

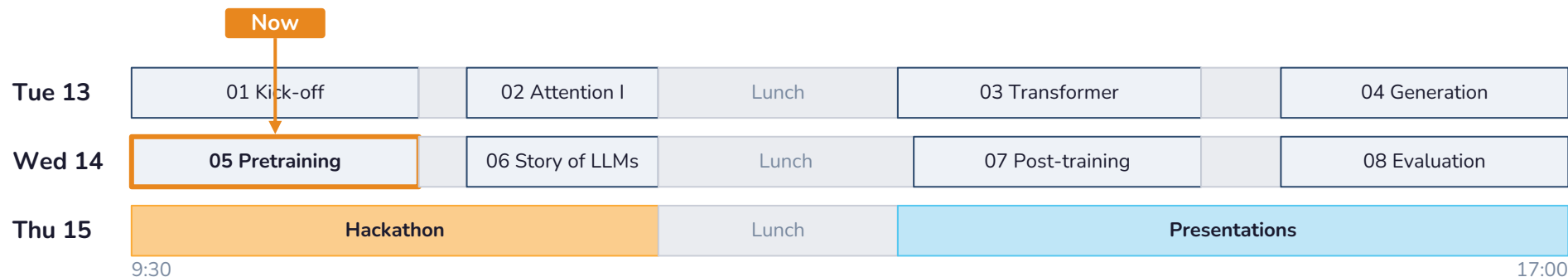
Advanced NLP

Session 05 · Pretraining

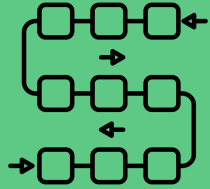
Prof. Dr. Richard Sieg · TH Köln IWS · WS 26 / 27
Wednesday, October 14, 2026

Twelve minutes of training, and the weights are yours

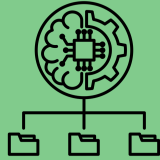
Time	
9:30	The data and the objective
9:46	The training loop
9:55	Lab: the training pieces
10:15	Your run, twelve minutes
10:45	Lab: talk to your own model, push it
11:00	Break



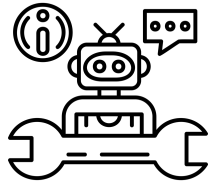
The second ingredient: efficient training on huge datasets



Process long sequences and context

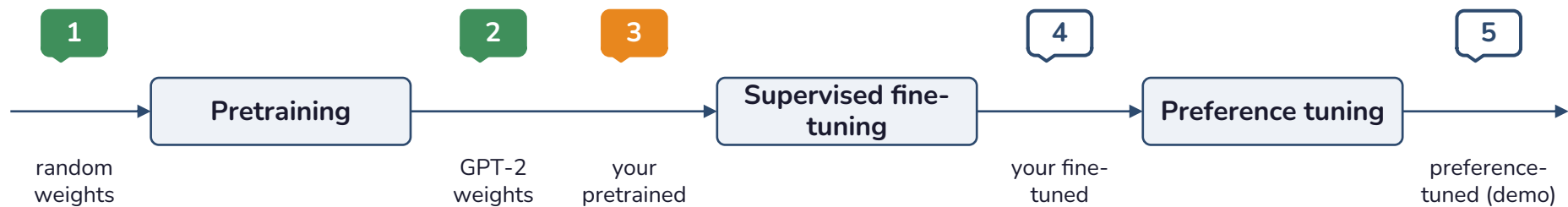


Efficient training on huge datasets



Follow (human) instructions

By 11:00 you have checkpoint 3



The loss measures how surprised the model is by the next token.

Agenda

01. The data

02. The objective

03. The training loop

04. Your run

01.

The data

Real base models read the web, cleaned hard

- Base models are trained mostly on text scraped from the web, then cleaned hard.
- Start from Common Crawl (snapshots of the web, billions of pages)
- Filter into curated corpora: C4, The Pile, Dolma, FineWeb
- Quality beats quantity: models use a data mix that upweights clean sources (Wikipedia, books) and downweights low-quality web text
- Standard steps: deduplication, removing boilerplate, basic safety filtering

We read children's stories instead, and that is enough

- **TinyStories:** synthetic short stories that only use words a typical three- or four-year-old understands, written by GPT-3.5 and GPT-4.
- Models under 10 million parameters trained on it write fluent, consistent stories over several paragraphs.
- Models the size of GPT-2 small, trained on general text, rarely stay coherent beyond a few words.
- Our training set: 541 million tokens of it, tokenized with GPT-2's tokenizer.

Data quality beats model size.

Sue sat down to eat her bread and butter in the park.
A bird flew down and sat next to Sue. The bird looked at her bread and said, "Can I have some?"

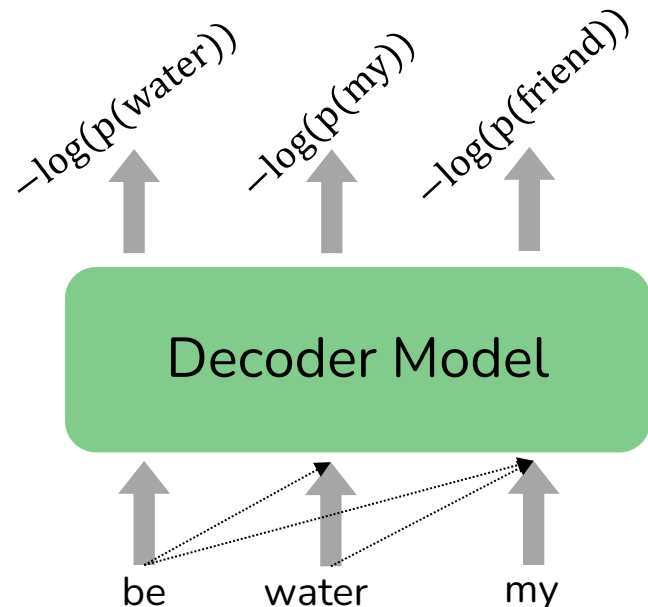


02.

The objective

Pretraining is next-token prediction

- How does the model learn to predict the next token? By reading a lot of text and guessing, over and over.
- **Self-supervised:** no human labels needed. The next word in the text is its own answer key
- **Objective:** at every position, predict the next token
- **Loss:** cross-entropy, the negative log probability the model gave to the true next word
- **Teacher forcing:** during training, always feed the correct history, not the model's own guess
- The result is a **base model:** very good at continuing text, trained on raw text alone



The loss is the average surprise at the next token

The model gave the true next token	Surprise, $-\log p$
Probability 1	0
Probability 0.5, a coin flip	0.69
Probability 0.1	2.30
Probability 1 / 50,257, a uniform guess	10.82

- Every position is one prediction. A batch of 64 windows of 512 tokens is 32,768 predictions, and the loss is their average.
- In code: flatten the logits from (64, 512, 50,257) to (32,768, 50,257) and the targets to 32,768, then `cross_entropy`.

Perplexity turns the loss into a number of choices

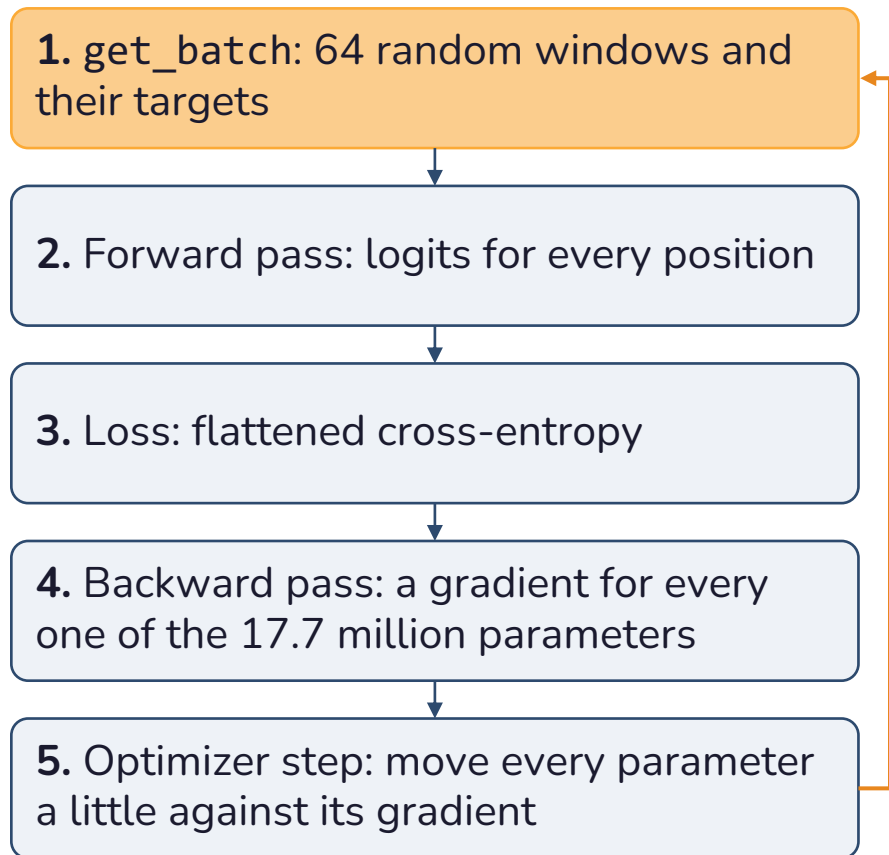
$$\text{Perplexity} = \exp(\text{loss})$$

- It reads as the number of tokens the model is choosing between at each step.
- Untrained: $\exp(10.82) = 50,257$, the whole vocabulary.
- After twelve minutes of training: you will see in an hour.

03.

The training loop

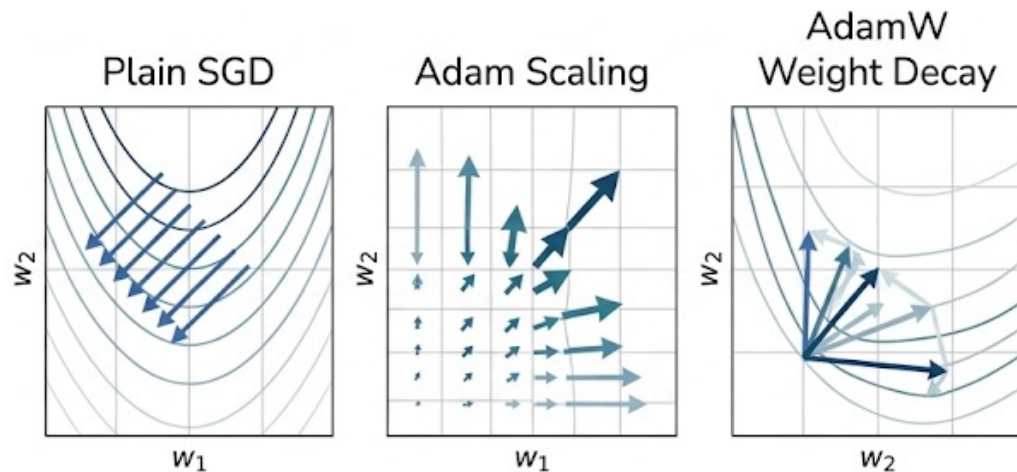
One training step is five lines



- Then repeat, about 4,700 times in twelve minutes.
- On the GPU the forward pass runs in bfloat16, which is faster and needs less memory, while the weights stay in full precision.

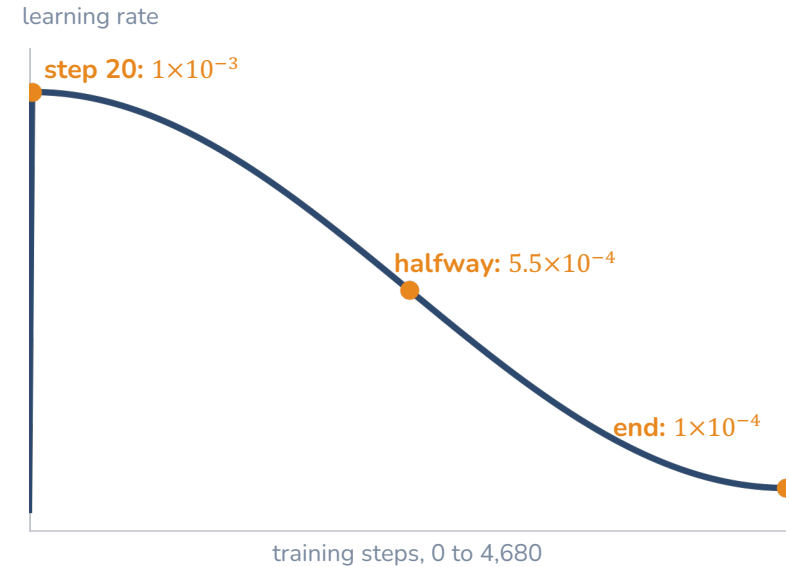
AdamW: a step size for every parameter

- Plain gradient descent moves every parameter by the same factor of its gradient.
- Adam keeps running averages of each parameter's gradients and of their squares, and scales each parameter's step by them.
- AdamW applies weight decay, a pull of every weight towards zero, separately from those averages, which regularizes better.
- Ours: learning rate up to 1×10^{-3} , weight decay 0.1, betas (0.9, 0.95).



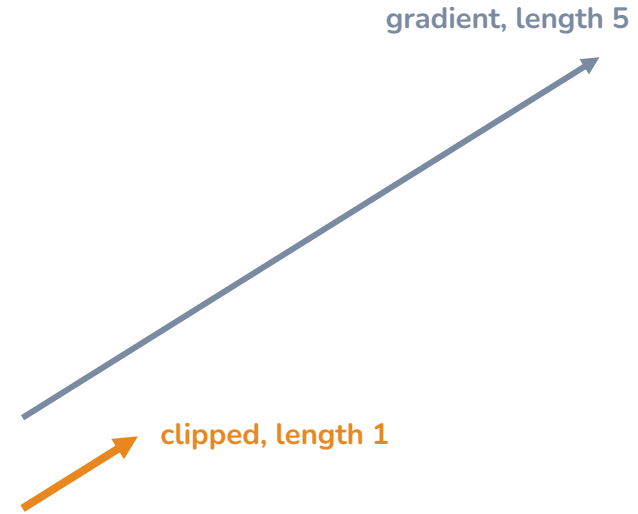
Warm up for 20 steps, then let the learning rate decay

- **Warmup:** start tiny and rise linearly to the peak over 20 steps, while the optimizer's averages are still empty and the gradients are wild.
- **Cosine decay:** then fall along half a cosine to a tenth of the peak, so the last steps are small and careful.
- Over our 4,680 steps: up to 1×10^{-3} by step 20, 5.5×10^{-4} halfway, 1×10^{-4} at the end.
- The notebook decays over wall-clock time rather than steps, so the schedule ends exactly at twelve minutes, however fast your GPU is.



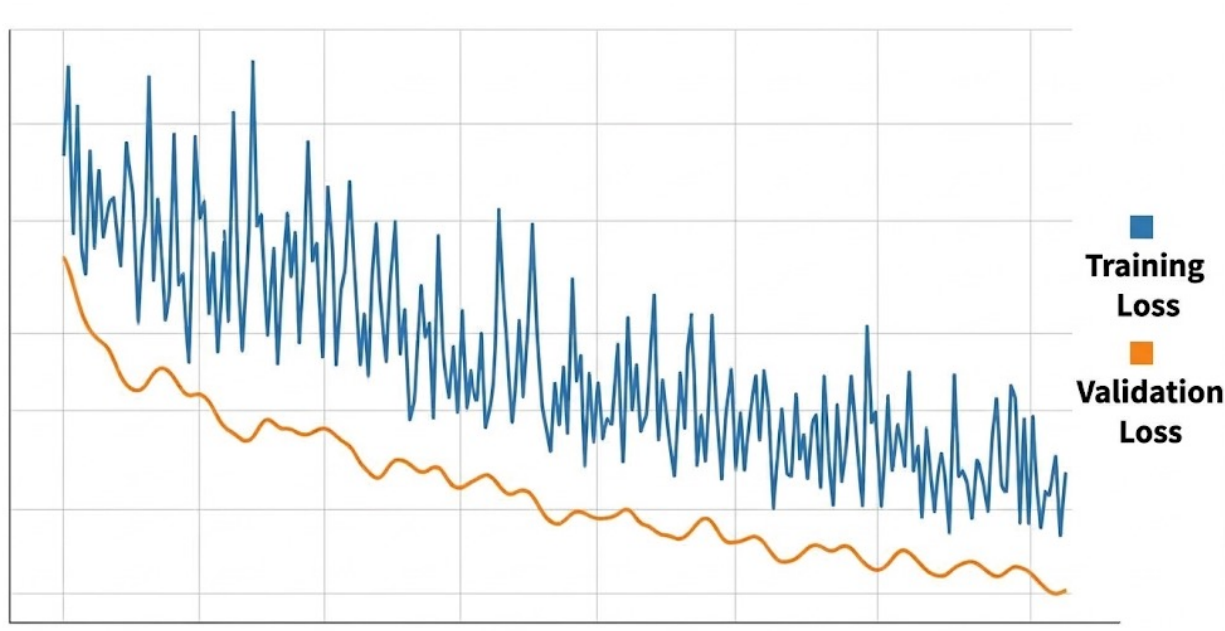
Clipping caps the push of one bad batch

- Before every step: if the gradient's overall length exceeds 1, scale the whole gradient down to length 1.
- Same direction, shorter step. A gradient of length 5 becomes one fifth of itself.
- One line in PyTorch, `clip_grad_norm_` with a maximum of 1.0.



Training loss above validation loss is fine here

- Training loss: one batch at a time, with dropout on, so noisy and slightly pessimistic.
- Validation loss: dropout off, averaged over the same 20 fixed batches every 100 steps.
- So the validation curve is smoother and a little lower. If it rises while training loss falls, something is wrong.



Tutorial

Notebook 05: the pieces of a training run

Until 10:20, in pairs

1. Implement `evaluate`. On the untrained model it should give about 10.8.
2. Implement `lr_at`: warmup by steps, then cosine decay by progress.
3. Implement `train_step`: forward, flattened cross-entropy, backward, clip, step.
4. When all three checks are green, press **Run pretraining**. Twelve minutes.
5. At 10:20, start the run either way: from your own notebook if it works, from the solution if not.

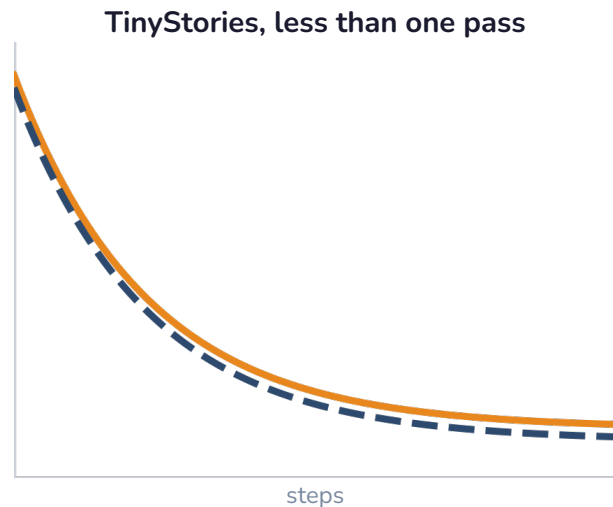
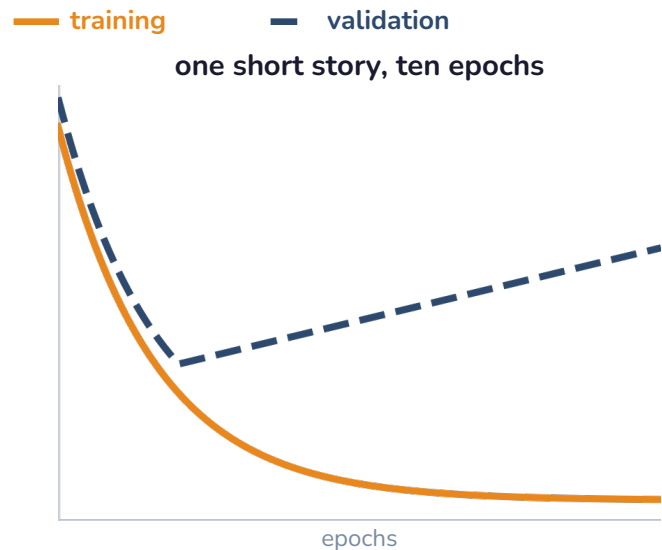
- `evaluate` gives about 10.8 on the untrained model
- `lr_at` passes its three checks
- Your run has started by 10:20

04.

Your run

Your model never sees a token twice

- Raschka's book trains on one short story for ten epochs. After two, training loss keeps falling, validation loss stalls, and the model starts reciting the story word for word.
- Twelve minutes covers 28 percent of our training data. Every window is new to the model.
- So training and validation loss fall together, as they should.



Every 250 steps, your model tells a story

- The notebook generates from “Once upon a time” every 250 steps, with yesterday's generation loop.
- First noise, then words, then sentences, then stories.
- In our reference run: fragments at step 250, sentences by 500, stories by about 1,000.

Once upon a time mildly
adject luxurycriminal mildly
originals reluctantlyyard
Theater educator('existent
americFair strategy
Alliedarianwordpress createEh
create create thing manufact
trucks partake underwear
trucks bragging and loser reb
balanced balanced besides
PTSD191anism\$. improvised
laundry indications
laundry138KB gad retention
Stargcli Graduate generates
unbeatenarian besides191
Britann vigilance requestsID

0

Once upon a time in aOnce
upon a time, there was aOnce
there lived a funny. This
lived in a small town.
TheOnce upon aOnceOnce upon a
time, there lived aOne day,
there lived aOnce upon a a
lonely girl named Lily. She
lived a happy, there. This
upon a nice friend, there was
a happy and play on a girl
named Lily. She was very much
fun. She loved to buy another
day. One day, she saw a
little girl named Lily went

250

Once upon a time, there was a
big tree. The tree loved to
hop all day long. One day,
the tree had a big tree. The
tree said, "I can play with
me?" The tree thought the
tree was a big tree to beak
and made a big tree to the
tree. The tree was sad and
wanted to help the tree. But,
something unexpected
happened. The tree was not a
strong, dark tree. The tree
was not strong anymore.

500

Once upon a time, there was a
little boy named Tim. Tim
loved to play with his toy
drum. One day, he saw a big
blue ball outside his window.
He wanted to play with it all
day. Tim went to his mom and
said, "Mom, can I play with
my drum?" His mom said, "Yes,
but be careful. The big blue
ball might break." Tim was
happy and started to play his
drum. He blew all the other
balls by himself, and they
all laughed.

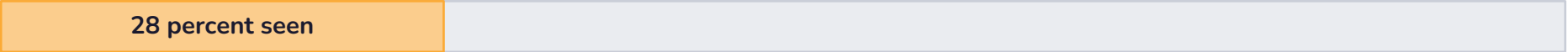
4500

Twelve minutes is a quarter of the data

- $4,680 \text{ steps} \times 64 \text{ windows} \times 512 \text{ tokens}$: 153 million tokens, 28 percent of the training set.
- One full pass would take about 42 minutes.
- A rule from 2022 says a model should see about 20 tokens per parameter. For our 17.7 million parameters, that is 355 million. We get 43 percent of it.

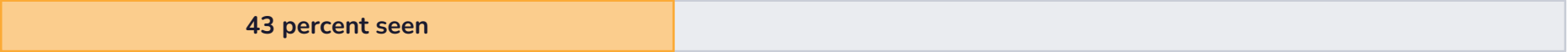
Training set, 541 million tokens

28 percent seen



20 tokens per parameter, 355 million

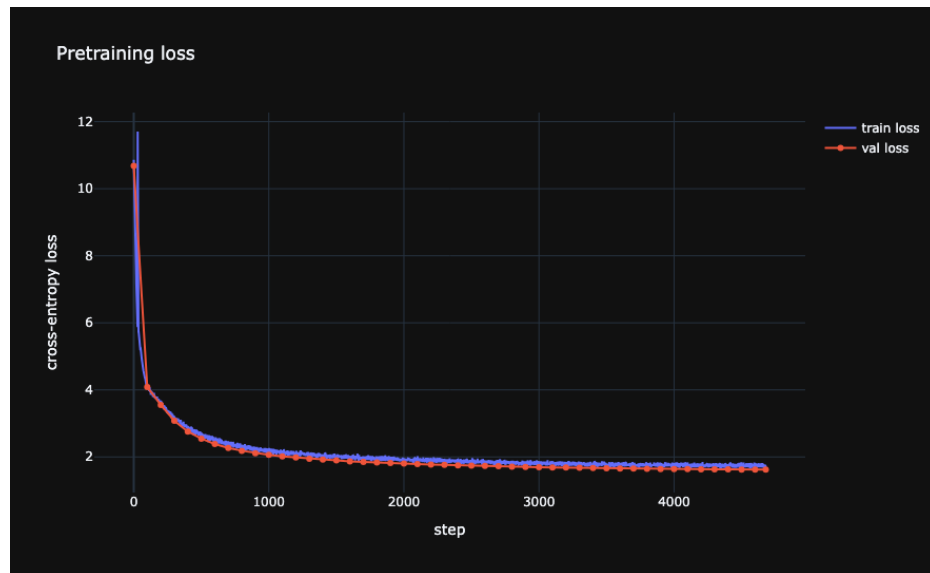
43 percent seen



Your model after twelve minutes

	Loss	Perplexity
Untrained	10.82	50,257
After 2,000 steps	1.88	6.5
After twelve minutes	about 1.6	about 5

From 50,257 choices to about five.



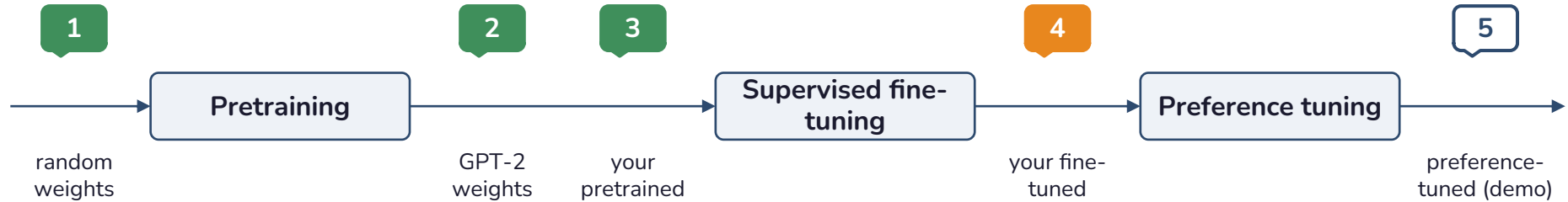
The loss measures how surprised the model is by the next token.

Checkpoint 3: talk to your own model, then push it

1. Open the chat window below the training results. Give it “Once upon a time”.
 2. Ask it a question. What does it do?
 3. Give it the start of a story with a name in it. Does it keep the name?
 4. The push cell has uploaded your checkpoint to a private repository under your Hugging Face name. Open it on the Hub.
- Your model tells stories
 - Your checkpoint is on the Hub, under your name

This afternoon, your model learns to follow instructions

- It now continues text, and only stories.
- After lunch: the story of how GPT-2 became ChatGPT, told as changes to your model.
- Then supervised fine-tuning turns your checkpoint 3 into checkpoint 4, a model that does what the prompt asks.



Sources

Literature

- Raschka, Build a Large Language Model (From Scratch), section 5.1: cross-entropy and perplexity. Section 5.2: the training loop, AdamW, overfitting on a small dataset. Appendix D: warmup, cosine decay, gradient clipping.
- Eldan, Li 2023, TinyStories, arxiv.org/abs/2305.07759.
- Loshchilov, Hutter 2019, Decoupled Weight Decay Regularization, arxiv.org/abs/1711.05101.
- Hoffmann et al. 2022, Training Compute-Optimal Large Language Models, arxiv.org/abs/2203.15556. The 20 tokens per parameter.